WSL 2 + Docker + Dorarion

> Beta baseline note (2026-10-10): current core installs two consolidated migrations. Replacing an earlier beta DB requires a new empty DB or an explicit reinstall that discards old data. Do not delete only the SQLx ledger. Use a fresh image built from this source. The README contains the current policy; an earlier downloaded PDF may describe the previous chain.

Simple local installation guide

> **Goal:** Install Ubuntu in WSL 2, install Docker, clone Dorarion, generate its required secrets, create the first administrator, and open Dorarion in your Windows browser.

- Run Windows commands in PowerShell as Administrator.
- Run Linux and Docker commands inside the Ubuntu WSL terminal.
- This guide follows the Dorarion Compose workflow used successfully on a local machine.
- A PDF version is also available.

1. Install WSL 2 and Ubuntu

Open PowerShell as Administrator and run:

```
wsl --install -d Ubuntu
```

Restart Windows if asked. Then open Ubuntu from the Start menu and create your Linux username and password.

Optional check from PowerShell:

```
wsl -l -v
```

Ubuntu should show `VERSION 2`.

2. Prepare Ubuntu

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git openssl ca-certificates curl
```

3. Install Docker Engine and Compose

3.1 Remove old or conflicting Docker packages

```
sudo apt remove -y docker.io docker-compose docker-compose-v2 \
  docker-doc docker-buildx podman-docker containerd runc
```

3.2 Add Docker's official repository

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

3.3 Install and test Docker

```
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

sudo docker run --rm hello-world
docker --version
docker compose version
```

The commands below use `sudo` with Docker. That is fine for this local installation and avoids Docker socket permission problems.

4. Download Dorarion

```
mkdir -p ~/projects
cd ~/projects
git clone https://github.com/dorarion/dorarion-compose.git
cd dorarion-compose
```

5. Prepare Dorarion

Run Dorarion's Compose installer once. It prepares the Compose environment, including `.env.docker`:

```
cp .env.docker.example .env.docker
chmod 600 .env.docker
${EDITOR:-nano} .env.docker
```

Check the environment file:

```
${EDITOR:-nano} .env.docker
```

If `.env.docker` is owned by root, and only if you get a permission error while editing the file, run:

```
sudo chown "$USER:$USER" .env.docker
```

6. Generate Dorarion secrets

Run these commands in the `dorarion-compose` directory:

```
POSTGRES_PASSWORD="$(openssl rand -hex 32)"
JWT_SECRET="$(openssl rand -hex 32)"
SECRET_ENCRYPTION_KEY="$(openssl rand -base64 32)"
chmod 600 .env.docker

sed -i "s|^POSTGRES_PASSWORD=.*|POSTGRES_PASSWORD=$POSTGRES_PASSWORD|" .env.docker
sed -i "s|^JWT_SECRET=.*|JWT_SECRET=$JWT_SECRET|" .env.docker
sed -i "s|^SECRET_ENCRYPTION_KEY=.*|SECRET_ENCRYPTION_KEY=$SECRET_ENCRYPTION_KEY|"
.env.docker
unset POSTGRES_PASSWORD JWT_SECRET SECRET_ENCRYPTION_KEY
```

Make sure no placeholder values remain:

```
if grep -q 'replace-' .env.docker; then echo 'Replace remaining image/credential
placeholders'; fi
```

The expected result is no output.

> **Keep `.env.docker` private.** It contains database and application secrets. Do not post it in chat, screenshots, tickets, or Git commits.

7. Start Dorarion

```
sudo ./scripts/install-compose.sh
sudo docker compose --env-file .env.docker ps
```

If the services are running, continue to the administrator setup.

8. Generate the first Dorarion administrator password

Create the first local administrator with username `admin` and email `admin@example.com`:

```
cd ~/projects/dorarion-compose
sudo docker compose --env-file .env.docker run --rm dorarion \
  bootstrap-admin admin admin@example.com
```

Save the generated administrator password immediately in a password manager. You will use it on the login page.

Field	Value
Username	`admin`
Email	`admin@example.com`
Password	The generated bootstrap administrator password

9. Open Dorarion

On Windows, open Chrome, Edge, or another browser and go to:

http://127.0.0.1:3000/login

You can also try:

http://localhost:3000/login

Sign in with `admin` (or `admin@example.com`) and the generated administrator password from Step 8.

10. Useful checks

```
sudo docker compose --env-file .env.docker ps
sudo docker compose --env-file .env.docker logs --tail=100
```

If Dorarion is not opening, first confirm that the containers are `Up` and that port `3000` is published.

Quick recap

- WSL 2 and Ubuntu installed.
- Docker Engine and Docker Compose installed.
- `dorarion-compose` cloned into `~/projects/dorarion-compose`.
- `.env.docker` secrets generated.
- Dorarion services started.
- First administrator generated with `bootstrap-admin`.
- Dorarion opened at http://127.0.0.1:3000/login.

Official platform references

- Install WSL
- Install Docker Engine on Ubuntu

Installation and upgrades — reviewed core

These settings describe the reviewed pre-1.0 core source. Select an image built from that source; an existing v0.0.3 registry tag does not prove it contains the remediation changes.

Pre-1.0: use the API contract shipped with the selected image. The reviewed source implements CI/CD, jobs, incidents and daily/weekly activity reporting.

- Generate `JWT_SECRET` (at least 32 bytes) and `SECRET_ENCRYPTION_KEY` (Base64 encoding of 32 random bytes) once. Keep both outside version control.
- The API pool defaults to 15 connections; allow another six worker connections per replica and temporary migration connections.
- Compose trusts only its exact internal `postgres` host. External PostgreSQL needs `sslmode=verify-full` and the trusted CA.
- Use `env://DORARION_EXT_<NAME>` for external credentials. Core secret variables are always denied.
- Argo CD needs a separately rotated application token and an active application/integration. Incident automation needs a repository-scoped incident source.

`/health/live` checks the process; `/health/ready` checks PostgreSQL. Worker health is separate at authenticated `/health/workers`. For Prometheus, store a 32–512-character ASCII `DORARION_METRICS_SCRAPE_TOKEN` in a Secret and use it only at `/metrics`. Configure exact trusted proxy networks before honoring forwarded client addresses.

Back up PostgreSQL and the exact encryption key before forward migrations. Run the migration Job/service once and disable server startup migrations. Change the generated bootstrap password on first login. Rotate Argo application tokens after upgrading. Audit retention cannot be shorter than 365 days. Weekly reports cover the seven complete local days before sending; reuse the same Idempotency-Key when retrying one manual send. Incident reopening must be explicit. Read the bundled API contract: demo routes were removed and unknown request fields are rejected. Validate backups, provider delivery and readiness in your environment before production rollout.

Compose: select `DORARION_IMAGE` before starting. Local HTTP uses `DORARION_COOKIE_SECURE=false`; set a reachable HTTPS public origin and secure cookies for remote use. The published port binds to `127.0.0.1` by default. `/health/ready` is database-only; the container uses the native `dorarion-core healthcheck` command. `DORARION_MIGRATE_ON_START=false` delegates migration to the one-shot service. The environment file is private (0600); avoid printing it.